EAST MEETS WEST · EMW-001

One Lattice, Three Addresses

State Unranking in Sanskrit Prosody and Zero-Allocation Integer Systems

Sean Morin (Deveraux)

2748684 Alberta Ltd., Edmonton, Alberta · deveraux.dev

DOI 10.5281/zenodo.23334659 · Code github.com/13forge/proofs

Draft v0.4, release candidate · 13 October 2026

Abstract

Sanskrit prosody needed to catalogue every possible metre. Over about fifteen centuries it produced three exact, table-free ways to go between an integer index and a metre: positional (binary), weight-constrained (binomial), and duration-constrained (Fibonacci). Modern enumerative combinatorics calls these *ranking* and *unranking*. This paper states the three schemes precisely, proves them, and checks them exhaustively by machine. It extends the weight-constrained scheme to balanced-ternary vectors with exactly r non-zero digits, and shows that the binomials every scheme needs can be built by Virahāṅka's *sūci-prastāra* as add-only prefix scans, which removes division from the whole path. It corrects several inflations of the historical record, including three errors in our own earlier drafts. It maps each scheme onto state addressing in a deterministic integer runtime, with measured bounds on where a table-free address actually pays. We do not claim descent from these texts to modern computing. The claim is narrower: anyone who must enumerate a discrete space under a hard constraint ends up on the same lattice, and the prosodists reached it from a linguistic constraint.

Keywords: Sanskrit prosody; Piṅgala; Virahāṅka; *prastāra*; *sūci-prastāra*; ranking and unranking; enumerative coding; combinatorial number system; prefix sum; balanced ternary; run-length-limited codes; deterministic computing.

Tag	Meaning
PROVEN	Statement with proof given here, plus exhaustive machine check (Appendix A)
BUILT	Running code exists with receipts
DESIGN_TARGET	Intended integration, not yet built
[VERIFY]	Historical claim that needs a primary or peer-reviewed source before release

Statements headed *Remark* are standard results given for orientation; they carry no tag and are not part of the machine check.

Text licensed CC BY 4.0. Code MIT OR Apache-2.0.

1 Scope and non-claims

1. **No lineage.** Modern computing descends from Leibniz → Boole → Shannon. Indian prosody is independent discovery, not ancestry.
2. **No hardware.** These are methods for enumerating and counting. There is no logic-gate design and no Boolean algebra.
3. **No "father of computing."** That framing is rejected.
4. **No priority on the bridge.** We know of no prior work that frames *sūci-prastāra* as a scan primitive. Absence of evidence is not proof of absence, so we make no "first" claim.
5. **No new mathematics.** Every theorem here is a known result or a direct composition of known results (Lehmer, Cover, Schalkwijk, Knuth). What is new is the reading: the correspondence, the corrections, and the receipts.
6. **What is claimed:** the same addressing structures appear in a premodern linguistic tradition, in exact algorithmic form, and they transfer without modification to a zero-allocation integer system.

2 Chronology (corrected)

Date	Source	Contribution	Basis
c. 3rd–2nd c. BCE	Piṅgala, *Chandaḥśāstra* 8.20–8.35	Terse sūtras: *prastāra* 8.20–23, *naṣṭa* 8.24–25, *uddiṣṭa* 8.26–27, *saṅkhyā* 8.28–31, *adhvayoga* 8.32–33, "*pare pūrṇam*" 8.34–35. The Yajur recension carries an extra *lagakriyā* sūtra ("*ekottarakramaśaḥ pūrvapṛktā lasaṃkhyā*") absent from Halāyudha's text.	Kedāranātha 1938; Kulkarni 2008; Shah 2008 pp. 8, 29
c. 1st c. CE	Bharata, *Nāṭyaśāstra*	***Lagakriyā* by repeated partial sums**, dropping the terminal sum each pass: the earliest firmly attested binomial construction in the Indian record.	Shah 2008 pp. 2, 9–10
c. 7th c. CE	Virahāṅka, *Vṛttajātisamuccaya* 6.7–6.12	Names and gives **both** constructions: ***sūci-prastāra*** ("needle spread", the partial-sum column method) and ***meru-prastāra*** (the two-cells-above triangle). Meru first appears here, not in Halāyudha. Also the *mātrā* rule: each count is the sum of the previous two.	Shah 2008 pp. 10, 19, 31–32 (tr. of Velankar 1962); Singh 1985
c. 950	Halāyudha, *Mṛtasañjīvanī*	First to read sūtra 8.34–35 as an instruction to build the meru. **Disputed**: Weber and Shah reject it, Alsdorf defends it. The triangle itself is three centuries older.	Shah 2008 pp. 2, 8, 29
1011–1077	Shao Yong	The *xiantian* ("Fuxi") hexagram arrangement, later read as a binary ordering; whether his own usage was binary is contested.	Maitre 2025

Date	Source	Contribution	Basis
c. 1100	Kedāra Bhaṭṭa, *Vṛttaratnākara*	The "write 1, 2, 4 over the syllables and add" form of *uddiṣṭa* usually quoted as Piṅgala's.	Kulkarni 2008 p. 14
before 1135	Gopāla (on Virahāṅka)	Worked values 3, 5, 8, 13, 21.	Singh 1985
c. 1150	Hemacandra	Same recurrence, stated independently.	Singh 1985
1202	Fibonacci, *Liber Abaci*	Same recurrence, in Europe.	—
early 14th c.	*Prākṛta Paiṅgala*	***Mātrā-meru***: *mātrā* metres tabulated by guru count; entries are binomial.	Singh 1985
1356	Nārāyaṇa Paṇḍita, *Gaṇita Kaumudī*	***Sūcī-paṅkti*** ("needle sequence"): a separate, product-built construction yielding multinomial generalisations. Not Virahāṅka's *sūci-prastāra*.	Singh 1985
15 Mar 1679	Leibniz, *De progressione dyadica* (MS)	Binary arithmetic, dated in Leibniz's hand.	Hochstetter & Greve 1966
4 Nov 1701	Bouvet → Leibniz (recv. 1 Apr 1703)	Hexagram woodcut, which Leibniz took as **confirmation**, not origin. Leibniz had sent Bouvet his binary table on 15 Feb 1701.	Leibniz–Bouvet archive
1703 (pr. 1705)	Leibniz, *Explication de l'Arithmétique Binaire*	Sent to the Académie within a week of Bouvet's letter arriving.	Leibniz–Bouvet archive
1967	Ingerman, *CACM* 10(3)	Proposes "Pāṇini–Backus Form".	Ingerman 1967

Corrections logged against circulating accounts.

- *"Leibniz got binary from the I Ching."* False. He had binary in 1679 and received the hexagrams in 1703 (letter dated 1701).
- *"Piṅgala wrote Pascal's triangle."* Disputed. The triangle first appears by name in Virahāṅka (7th c.); Halāyudha (10th c.) is the first to attribute it to Piṅgala's sūtra, and Shah argues the sūtra does not support that. Shah does trace the *computation* of binomials to Piṅgala via the Yajur-recension *lagakriyā* sūtra.
- *Sūci-prastāra.* The **name** is Virahāṅka's (7th c.). The **construction** is already in Bharata's *Nāṭyaśāstra* (c. 1st c. CE), which holds the earliest firm attestation. Our v0.1 draft said "earliest … earlier than the meru"; both halves were wrong, since Virahāṅka gives sūci and meru side by side. Nārāyaṇa's *sūcī-paṅkti* (1356) is a different object.
- *Naṣṭa orientation* (*our own v0.1 error*). Sūtra 8.24 *l-ardhe*: "if it halves, L"; 8.25 *sa-eke g*: "with one added, G". So an **even** row index yields L and an **odd** one yields G, row 1 being all-G. Our draft had the parities swapped. The "write 1, 2, 4 and add" form of *uddiṣṭa* is Kedāra Bhaṭṭa's, not Piṅgala's 8.26–27, which doubles from the first laghu and subtracts one at each guru. Corrected in §3–§4.
- *"Oldest prefix-sum accumulator in history."* Unsupported superlative. Removed.
- *"Prosodists flattened the triangle to cut working-memory load."* A plausible motive, but no text says it. Removed.

3 Encoding

A metre of n syllables is a word in $\{\mathrm{L},\mathrm{G}\}^n$. Laghu (L, short) $\mapsto 1$, guru (G, long) $\mapsto 0$. Syllable i (0-indexed from the start of the line) is bit i, so **the first syllable is the least significant bit**. This is the convention under which Piṅgala's row k reads as the binary of $k-1$ (Van Nooten 1993; Kulkarni 2008 §2.2.1). In *mātrā* (duration) metres, L weighs 1 and G weighs 2.

The same construction works in any radix B: Kulkarni (2008, Table 11) states Piṅgala's *uddiṣṭa* for base B directly. A balanced-ternary lattice (digits $-1, 0, +1$ stored as $0, 1, 2$) is the $B = 3$ case, and its pack/unpack is *naṣṭa*/*uddiṣṭa* with 3 in place of 2.

4 Addressing I: positional (*prastāra*, *naṣṭa*, *uddiṣṭa*)

Prastāra. The canonical enumeration of $\{\mathrm{L},\mathrm{G}\}^n$ starts at all-G. Each next row flips the first G to L and resets every earlier syllable to G: a binary counter running backwards. Indices are **1-based**.

Naṣṭa (index → metre), sūtras 8.24–8.25. Given row k: if k is even (*l-ardhe*), the syllable is L and $k \leftarrow k/2$. If k is odd (*sa-eke g*), the syllable is G and $k \leftarrow (k+1)/2$. Repeat n times.

Uddiṣṭa (metre → index), sūtras 8.26–8.27. Piṅgala: start at 1 on the last syllable and work backwards, doubling at each syllable and subtracting one after doubling at each guru. Kedāra Bhaṭṭa's later form is the one usually quoted: write $1, 2, 4, \ldots$ over the syllables, add the values over the **laghus**, then add 1. Both give the same number.

Theorem 1 (Positional address — **PROVEN**)**.** *Under the encoding of §3, naṣṭa$(k) = k-1$, and uddiṣṭa is its inverse. The prastāra is the bijection $\{1,\ldots,2^n\} \leftrightarrow \{0,1\}^n$ given by the little-endian binary of $k-1$.*

Proof. Write $k-1 = \sum_i b_i 2^i$. Then k even $\iff$ $b_0 = 1$ $\iff$ L. Both update rules map k to $1 + \lfloor (k-1)/2 \rfloor$, which shifts $k-1$ right by one bit. By induction on n, the i-th emitted syllable is b_i. Kedāra's *uddiṣṭa* computes $1 + \sum_i b_i 2^i = k$ directly; Piṅgala's backward doubling is Horner's rule on the same digits. □

Saṅkhyā (count). $|\{\mathrm{L},\mathrm{G}\}^n| = 2^n$, computed by Piṅgala's rule: for even n, halve and square; for odd n, subtract one and double. This is exponentiation by squaring, $O(\log n)$, the method modular exponentiation in RSA uses.

Cost. $O(n)$ shifts and masks, no table, no allocation. The halvings in *naṣṭa* are shifts.

5 Addressing II: weight-constrained (*lagakriyā*, *sūci*, *meru*)

The number of n-syllable metres with exactly r laghus is $\binom{n}{r}$. *Lagakriyā* ("the L-reckoning") computes these values; the *sūci* and *meru prastāra*s tabulate them.

Theorem 2 (Weight-constrained address — **PROVEN**)**.** *The weight-r rows of the prastāra are addressable by the **combinatorial number system**: every $0 \le k < \binom{n}{r}$ has a unique representation $k = \sum_{j=1}^{r} \binom{c_j}{j}$ with $n > c_r > \cdots > c_1 \ge 0$, and the laghu positions are $\{c_j\}$. Unranking is greedy: for $j = r$ down to 1, take the largest c with $\binom{c}{j} \le k$, set laghu c, and subtract.*

Proof. Standard (Lehmer; Knuth, TAOCP 7.2.1.3). Exhaustively checked for $n \le 20$ (Appendix A). □

Cost. $O(n)$ binomial lookups. In the reference implementation the binomials come from a compile-time table built by the *sūci-prastāra* of §5.1 (add-only; 65×65 entries, every $\binom{n}{k}$ with $n \leq 64$), so the unranking path contains no division. The multiplicative formula, which divides, is kept only as a test oracle for that table.

5.1 Sūci-prastāra: the binomials as an iterated scan

Bharata, and after him Virahāṅka under the name *sūci-prastāra*, build the binomial columns by partial sums. Column 1 is $1, 2, \ldots, n$. Each later column is the running sum of the one before, with the last entry dropped (Shah 2008 pp. 9–10, 31–32; Virahāṅka 6.7–6.9: "one-by-one, add the number above ... successively leave out the last number").

Theorem 3 (Hockey-stick = one *sūci* pass — **PROVEN**). $\sum_{i=r}^{n} \binom{i}{r} = \binom{n+1}{r+1}$. *Equivalently, applying an inclusive scan k times to the all-ones vector gives $\binom{i+k}{k}$ at index i.*

Proof. Pascal's rule $\binom{i+1}{r+1} = \binom{i}{r+1} + \binom{i}{r}$ telescopes. Induct on k for the scan form. Checked for $r \leq 20$, $n \leq 60$ (identity form) and $k \leq 16$, $i < 48$ (scan form). □

Remark (Matrix form). One inclusive scan of a length-m vector is multiplication by the lower-triangular all-ones matrix L. So k *sūci* passes are multiplication by L^k, and $L^k[i][j] = \binom{i-j+k-1}{k-1}$ for $i \geq j$. The *sūci-prastāra* is a power of a triangular matrix whose entries are binomials, which is how scan is mapped onto matrix units in practice (Dakkak et al. 2019). Checked outside the crate for $k \leq 8$, $m = 12$; not a **PROVEN** item.

Systems reading. The claims below are bounded on purpose.

Property	Sūci (iterated scan)	Meru (row recurrence)	Multiplicative $\binom{n}{k}$
Ops, column of length n, order k	nk adds	nk adds (in-place, right-to-left)	$O(k)$ multiplies **and divides** per entry
Working state	one accumulator + caller buffer	caller buffer	$O(1)$
Arithmetic	**add-only**	add-only	needs division
Maps to	standard scan primitive (Blelloch 1990): SIMD, GPU, FPGA accumulator	2-tap stencil	scalar ALU

1. Sūci and meru do the **same work** with the **same asymptotics**. A properly written row update is also a sequential single-buffer pass. The claim that meru is "cache-thrashing" does not hold up and is not made here.
2. Sūci's real systems property is that **each pass is exactly a prefix scan**, the most heavily optimised parallel primitive there is. Binomial tables inherit $O(\log n)$-depth parallel scans and existing hardware scan units with no new kernels.
3. Sūci is **add-only and division-free**. In an integer-deterministic runtime, that removes the only division in the binomial path.
4. Bench, as a **DESIGN_TARGET**: iterated SIMD scan vs in-place row update vs multiplicative formula, for $n \leq 4096$ and $k \leq 64$. The hypothesis is parity between the first two; the point is the division-free result, not raw speed.

5.2 Signed weight: exactly r non-zero digits in $\{-1, 0, +1\}^n$

Split a balanced-ternary vector into its **support** s (bit i set iff digit $i \neq 0$) and its **sign pattern** $p \subseteq s$ (bit i set iff digit $i = +1$). Let $r = |s|$.

Theorem 4 (Signed-weight address — **PROVEN**)**.** *For fixed n and r, the map*

$$\mathrm{rank}(s, p) = \mathrm{crank}(s) \cdot 2^r + \mathrm{pext}(p, s)$$

is a bijection from the vectors with exactly r non-zero digits onto $\left[0, \binom{n}{r}2^r\right)$, where crank *is the combinadic rank of Theorem 2 and* $\mathrm{pext}(p, s)$ *packs the bits of p at the positions of s into an r-bit integer. The inverse is $s = \mathrm{cunrank}(\lfloor k/2^r \rfloor)$, $p = \mathrm{pdep}(k \bmod 2^r, s)$. The classes $r = 0, \ldots, n$ partition the 3^n vectors.*

Proof. Theorem 2 gives the bijection supports $\leftrightarrow \left[0, \binom{n}{r}\right)$. For a fixed support, pext/pdep are mutually inverse bijections between sign patterns and $[0, 2^r)$. Composing by the mixed radix $\left(\binom{n}{r}, 2^r\right)$ gives the result, and $\sum_r \binom{n}{r}2^r = 3^n$ by the binomial theorem. Exhaustively checked over all $3^{13} = 1\,594\,323$ vectors and at every width $n \leq 9$. □

This is enumerative coding (Cover 1973; the fixed-weight case is Schalkwijk 1972) under the lexicographic order (support, then sign pattern), applied to a ternary constant-weight set. It is a re-derivation, not a new method. The division is again absent: $\lfloor k/2^r \rfloor$ and $k \bmod 2^r$ are a shift and a mask, and the binomials are the §5.1 table. pext/pdep are the BMI2 instructions on x86-64; the reference implementation uses a portable $O(r)$ loop by default and the instructions only when the build enables them, and checks the two bit-identical. The address fits a `u64` for $n \leq 40$ ($3^{40} < 2^{64} < 3^{41}$).

6 Addressing III: duration-constrained (*mātrā*)

Let $M(n)$ be the number of L/G sequences of total duration n, with L = 1 and G = 2. The first syllable is either L, leaving $n - 1$, or G, leaving $n - 2$. So $M(n) = M(n-1) + M(n-2)$ with $M(0) = M(1) = 1$, and therefore $\mathbf{M(n) = F_{n+1}}$. This is Virahāṅka's rule.

Theorem 5 (Mātrā address — **PROVEN**)**.** *Order sequences lexicographically with L < G. For a 0-indexed k and remaining duration n: if $k < M(n-1)$, emit L and set $n \leftarrow n - 1$; otherwise emit G, set $k \leftarrow k - M(n-1)$ and $n \leftarrow n - 2$. This is a bijection $\{0, \ldots, F_{n+1} - 1\} \leftrightarrow$ sequences of duration n, and the corresponding ranking is its inverse.*

Proof. Exactly $M(n-1)$ sequences begin with L, and every one of them precedes every sequence beginning with G. Induct on n. Checked against brute-force lexicographic enumeration for $n \leq 20$. □

Theorem 6 (Bridge identity: shallow diagonals — **PROVEN**)**.** *A duration-n sequence with g gurus has $n - 2g$ laghus, $n - g$ syllables, and $\binom{n-g}{g}$ arrangements. Hence*

$$F_{n+1} = \sum_g \binom{n-g}{g}.$$

The Fibonacci count is the binomial count refined by guru number: the shallow-diagonal sum of the binomial triangle, which is the content of the Prākṛta Paiṅgala mātrā-meru (Singh 1985).

Proof. Choose which g of the $n - g$ syllables are gurus. Checked for $n \leq 64$ (the extent of the binomial table); per-guru counts checked for $n \leq 20$. □

Fibonacci in $O(\log n)$. Fast doubling: $F_{2m} = F_m(2F_{m+1} - F_m)$ and $F_{2m+1} = F_m^2 + F_{m+1}^2$. This is the *saṅkhyā* squaring pattern applied to a different recurrence.

Cost. $O(n)$ steps, one $O(\log n)$ Fibonacci evaluation per step (or a 94-entry constant table, since F_{93} is the largest Fibonacci number that fits in a `u64`). No allocation.

Remark (The *mātrā* constraint is a recording-channel constraint). Write a duration-n sequence as n unit morae with a mark in each of the $n-1$ gaps: 1 if the two morae are joined into one guru, 0 if they are separate. A valid sequence never joins three morae, so the gap string never has two adjacent 1s. This is a bijection between *mātrā* metres of duration n and binary strings of length $n-1$ with no two adjacent 1s: the $d = 1$ run-length-limited constraint of magnetic and optical recording (Marcus, Roth & Siegel 1998). Its information rate is $\log_2 \varphi \approx 0.694$ bits per mora (Shannon 1948), against 1 bit per syllable for the unconstrained positional metre. One search (13 October 2026) found no source identifying Virahāṅka's metres with the RLL constraint; this is a reading, not a priority claim.

7 One lattice: the addressings compared

Constraint	Count	Unrank method	Prosodic source
fixed length n	2^n	binary digits of $k-1$	Piṅgala 8.24–8.27: *naṣṭa*/*uddiṣṭa*
fixed length n, r laghus	$\binom{n}{r}$	greedy on binomials	*lagakriyā* (Bharata); *sūci-* and *meru-prastāra* (Virahāṅka)
fixed length n, r non-zero signed digits	$\binom{n}{r}2^r$	greedy on binomials, then sign bits	(extension, §5.2)
fixed duration n	F_{n+1}	greedy on Fibonacci	Virahāṅka; *mātrā-meru* (*Prākṛta Paiṅgala*)

Each row is a numeration system whose place values are the counts of the constrained space (powers of two, binomials, Fibonacci numbers). Unranking means writing k in that system.

Remark (The lattice, stated precisely). Every row is path counting in a small finite automaton: the unconstrained metre is one state with two edges; the *mātrā* metre tracks the remaining duration; the weight constraint tracks the laghu count. In each case the count is a sum of entries of a matrix power A^n (the transfer-matrix method; Stanley, EC1 §4.7), and unranking is the greedy walk against path counts that Cover (1973) calls enumerative coding. *Saṅkhyā*'s halve-and-square is A^n by repeated squaring, and the *sūci-prastāra* is L^k (§5.1). This is the sense in which the addressings are one lattice. They are not isomorphic to each other; they are instances of one construction.

8 Engineering mapping

Use	Scheme	Why it fits	Status
Deterministic state #k with no lookup table	I	Index → state in $O(n)$ integer ops; nothing allocated. **Bounded, two numbers**: at one byte of state ($3^5 = 243$ codes), a 64 KB pair-distance table against register decode on a desktop x86 core (Zen 3), working sets 32 KB to 2 MB, warm and after a 64 MB evict alike. **Throughput** (independent ops): table 0.8–1.6 ns/op vs decode 6.9–9.6, 6–9×. **Latency** (each input derived from the previous result): table 3.5–4.9 ns/op vs decode 9.1–10.5, 2.0–2.6×. An explicit multiply-shift decode is within noise of plain decode in both (6.6–7.7 throughput, 8.7–9.4 latency). The decode cost is the dependent chain of digit extractions, not the divide: the compiler already strength-reduces a constant divisor. Below about 16 K ops after an evict the table loses in latency (re-warming 64 KB is not amortised). Table-free wins only where a table cannot live: wide states (3^{13} and up), register-only GPU kernels, `no_std` targets without a data cache.	**DESIGN_TARGET** (bench measured; harness in repo, App. A)
Exactly r active of n (voices, channels, capability bits)	II	Dense index over valid states only; invalid states cannot be addressed at all.	**DESIGN_TARGET**
Exactly r non-zero of n **signed** digits	II×I	Theorem 4. **Re-derivation** of enumerative coding (Cover 1973; Schalkwijk 1972) for a ternary constant-weight set; the refinement by sign count $\binom{n}{r}\binom{r}{p}$ is the constant-composition case and the multinomial Nārāyaṇa's *sūcī-paṅkti* counts. At one byte ($n = 5$) the 243 (r, rank) pairs are a table generated at compile time *from* the formula: the formula is the generator and validator, the table is the hot path.	**PROVEN** (3^{13} exhaustive); integration **DESIGN_TARGET**
Rhythm space under a fixed bar length	III	Enumerates every subdivision of n ticks into 1- and 2-tick units, densely and in order.	**DESIGN_TARGET**
Division-free binomial tables	II (*sūci*)	Iterated scans; add-only integer path; maps onto SIMD scan. Built as a compile-time table in the reference implementation (§5).	**BUILT** (reference); SIMD **DESIGN_TARGET**
Sequence acceptance	I–III	A *sequence* (a metre, a *mātrā* string, a support mask) is well-formed iff ranking it and unranking the result gives it back. The rank round-trip is an acceptance test for the **input alphabet**, nothing more.	**DESIGN_TARGET**
ID integrity	I–III	A dense ID carries zero redundancy by construction: every integer in $[0, \text{count})$ is some valid state, so a flipped bit decodes to a valid *wrong* state and no round-trip can notice. Integrity in flight needs a check field or reserved capacity added on top; the addressing itself gives none.	**DESIGN_TARGET**

Prior-art search for the signed-weight row (13 October 2026, query "ternary constant-composition code enumerative ranking"): the hits are existence and classification results for ternary constant-composition codes (Bogdanova & Kapralov 2003; Chee, Ge & Ling; Gao & Ge), none on ranking or unranking. The search was not empty, so no candidate claim is made; the row stays a re-derivation of Cover/Schalkwijk.

The reference implementation (Appendix A) is `no_std`, integer-only, and allocation-free. The remaining rows are promoted to **BUILT** once it is lifted into a production workspace with benches.

9 Thesis

Piṅgala, Shao Yong, Leibniz and Shannon arrived at binary enumeration independently. Virahāṅka, Gopāla, Hemacandra and Fibonacci arrived at the *mātrā* recurrence independently. The binomial triangle appears in India, Persia, China and Europe. These recurrences are not cultural borrowings. They are what you get when you have to count a discrete space under a hard constraint, and they are found again because they are forced by that problem rather than chosen. The Indian record is notable for three reasons: the constraint was linguistic, the methods were algorithmic rather than tabular, and the inverse map (index → object) was stated explicitly. That inverse map is the part a zero-allocation system needs.

10 Open items before release

1. **[VERIFY]** Velankar's 1962 edition of the *Vṛttajātisamuccaya* not consulted directly; verses 6.7–6.12 are read in Shah's translation.
2. **[VERIFY]** Publisher imprint of Kāvyamālā 91.
3. **[VERIFY]** Weber's date: Shah gives 1835 in prose and 1863 in the reference list.
4. *Resolved* (was: the §8 row 1 bench harness lived outside the code repository): a self-contained copy is `prastara/examples/byte_pair_distance_bench.rs`; the Appendix A numbers are from that copy.
5. DOI reserved (10.5281/zenodo.23334659); the Zenodo record is published together with the release tag.

References

[1] Piṅgala. *Chandaḥśāstram*, with Halāyudha's *Mṛtasañjīvanī* and Madhusūdana's *Chandonirukti*. Ed. Kedāranātha. Kāvyamālā 91, Bombay, 1938. **[VERIFY]** publisher imprint not seen; commonly given as Nirnaya Sagar Press.

[2] Virahāṅka. *Vṛttajātisamuccaya*, with Gopāla's commentary. Ed. H. D. Velankar. Rajasthan Oriental Research Institute, Jodhpur, 1962 (Rajasthan Puratana Granthamala 61). **[VERIFY]** not consulted directly.

[3] J. Shah. A history of Piṅgala's combinatorics. Preprint, 2008, 44 pp. Published as *Gaṇita Bhāratī* 35(1–2), 2013, 1–54. Page numbers in this paper refer to the preprint.

[4] A. Kulkarni. Recursion and combinatorial mathematics in Chandaśśāstra. arXiv:math/0703658v2 [math.HO], 2008.

[5] A. Weber. *Über die Metrik der Inder.* Berlin, 1863 (Indische Studien 8). **[VERIFY]** date.

[6] L. Alsdorf. Die Pratyayas. Ein Beitrag zur indischen Mathematik. *Zeitschrift für Indologie und Iranistik* 9 (1933), 97–157. English tr. S. R. Sarma, *Indian Journal of History of Science* 26(1), 1991.

[7] B. Van Nooten. Binary numbers in Indian antiquity. *Journal of Indian Philosophy* 21(1) (1993), 31–50.

[8] P. Singh. The so-called Fibonacci numbers in ancient and medieval India. *Historia Mathematica* 12(3) (1985), 229–244. doi:10.1016/0315-0860(85)90021-7.

[9] Raja Sridharan, R. Sridharan, M. D. Srinivas. Nārāyaṇa Paṇḍita's enumeration of combinations and associated representation of numbers as sums of binomial coefficients. *Indian Journal of History of Science* 47 (2012), 607–631.

[10] T. Kusuba. *Combinatorics and Magic Squares in India: A Study of Nārāyaṇa Paṇḍita's Gaṇitakaumudī, Chapters 13–14.* PhD thesis, Brown University, 1993.

[11] B. Datta, A. N. Singh. *History of Hindu Mathematics.* 1935–38.

[12] M.-J. Maitre. Is the *Fuxi liushisi gua fangwei* diagram attributed to Shao Yong binary? *Science in Context*, 2025.

[13] G. W. Leibniz. *De progressione dyadica.* MS dated 15 March 1679, Niedersächsische Landesbibliothek, Hannover. Facsimile: Hochstetter & Greve, 1966.

[14] G. W. Leibniz. Explication de l'Arithmétique Binaire. *Mémoires de l'Académie Royale des Sciences*, 1703 (printed 1705). Leibniz–Bouvet correspondence: `https://archive.swarthmore.edu/leibniz-bouvet`.

[15] P. Z. Ingerman. "Pāṇini–Backus Form" suggested. *Communications of the ACM* 10(3) (1967), 137. doi:10.1145/363162.363165.

[16] C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal* 27 (1948), 379–423, 623–656.

[17] J. P. M. Schalkwijk. An algorithm for source coding. *IEEE Transactions on Information Theory* 18(3) (1972), 395–399.

[18] T. M. Cover. Enumerative source encoding. *IEEE Transactions on Information Theory* 19(1) (1973), 73–77.

[19] B. H. Marcus, R. M. Roth, P. H. Siegel. Constrained systems and coding for recording channels. In V. S. Pless, W. C. Huffman (eds.), *Handbook of Coding Theory*, Elsevier, 1998.

[20] R. P. Stanley. *Enumerative Combinatorics*, Vol. 1, §4.7 (the transfer-matrix method). Cambridge University Press, 2nd ed., 2012.

[21] G. Bogdanova, S. Kapralov. Enumeration of optimal ternary constant-composition codes. *Problems of Information Transmission* 39(4), 2003. doi:10.1023/B:PRIT.0000011273.98799.a8.

[22] G. E. Blelloch. Prefix sums and their applications. Technical report CMU-CS-90-190, 1990.

[23] A. Dakkak, C. Li, J. Xiong, I. Gelado, W.-m. Hwu. Accelerating reduction and scan using tensor core units. *Proc. ACM International Conference on Supercomputing (ICS '19)*, 2019.

[24] D. E. Knuth. *The Art of Computer Programming*, Vol. 4A, §7.2.1.3 (combinations) and §7.2.1.7 (history of generation).

A Reference implementation and receipts

`prastara/src/lib.rs` in github.com/13forge/proofs is `no_std` with no `alloc` crate (the test module alone uses `std`). Outside the test module it contains no division instruction: the halvings in *naṣṭa* are shifts, and the binomials come from the compile-time *sūci* table. It carries sixteen exhaustive tests, run with `cargo test --release` from the repository root. In the code, bit $i = 1$ means syllable i is **laghu**, per §3; the `nashta` rule "k even $\to$ set bit" is sūtra 8.24 read literally.

Test	Covers	Range
`nashta_is_binary_of_k_minus_1`	Thm 1	$n \le 20$, all k
`uddishta_inverts_nashta`	Thm 1	$n \le 20$, all k
`sankhya_is_pow2`	§4 *saṅkhyā*	$n \le 63$
`row_with_r_laghus_counts_binomial`	§5 count	$n \le 20$
`combinadic_is_bijection_onto_weight_r`	Thm 2	$n \le 20$, all r, all k
`suci_table_matches_multiplicative_binom`	§5 table	$n \le 64$, all k (`u128` oracle)
`hockey_stick_column_prefix_sum`	Thm 3, identity	$r \le 20$, $n \le 60$
`suci_iterated_scan_is_binomial`	Thm 3, scan	$k \le 16$, $i < 48$
`pext_pdep_hardware_matches_portable`	Thm 4, BMI2 = portable	all 12-bit (x, mask) pairs at offsets 0 and 52; all 3^{13} (sign, support) pairs
`pext_pdep_dispatch_matches_portable`	Thm 4, dispatch	all 3^{13}
`signed_weight_collapse_3_pow_13`	Thm 4	all 1 594 323 states: round-trip, bound, every class covered once
`signed_weight_collapse_small_widths`	Thm 4	every $n \le 9$
`signed_max_n_is_the_u64_ceiling`	Thm 4, `u64` bound	$3^{40} < 2^{64} < 3^{41}$
`matra_unrank_matches_brute_force_lex`	Thm 5	$n \le 20$, vs brute force
`shallow_diagonal_is_matra_count`	Thm 6	$n \le 64$
`matra_by_guru_count_is_binomial`	Thm 6, per guru	$n \le 20$

Receipts. 13 October 2026, Windows 11 x86_64 (AMD Ryzen 7 5800X3D), all exit 0.

Toolchain	Build	Result	Wall
stable rustc 1.95.0	`cargo +stable test --release`	16 passed; pext dispatch = portable	0.45 s
beta rustc 1.100.0-beta.4	`cargo test --release`	16 passed; pext dispatch = portable	0.40 s
beta rustc 1.100.0-beta.4	`RUSTFLAGS=-C target-feature=+bmi2 cargo test --release`	16 passed; pext dispatch = BMI2	0.37 s
beta rustc 1.100.0-beta.4	`cargo build --release --target wasm32-unknown-unknown`	`no_std` library builds	25.7 s

Bench behind §8 row 1. Harness: `prastara/examples/byte_pair_distance_bench.rs`, self-contained, no dependencies. Run with

```
cargo +stable run --release --example byte_pair_distance_bench
```

LUT = 64 KB pair table built at compile time from DEC; DEC = register decode by successive digit extraction (`/ 3`, `% 3`); MUL = explicit multiply-shift decode. TP = throughput (independent

ops); LAT = latency (each input derived from the previous result); cold = after a 64 MB evict. Stable rustc 1.95.0 (59807616e), ns/op, 6 reps per cell, same machine as the tests, exit 0.

Items	**TP warm** LUT / DEC / MUL	**TP cold** LUT / DEC / MUL	**LAT warm** LUT / DEC / MUL	**LAT cold** LUT / DEC / MUL
1 024	2.00 / 7.96 / 8.37	7.28 / 7.78 / 7.45	10.99 / 9.33 / 8.69	21.83 / 9.64 / 9.00
16 384	1.02 / 7.96 / 7.67	1.65 / 9.56 / 7.03	3.89 / 9.23 / 8.74	4.88 / 9.86 / 9.03
24 576	0.96 / 7.15 / 9.33	1.28 / 7.30 / 6.68	4.79 / 10.47 / 8.89	3.93 / 9.42 / 8.66
40 000	0.87 / 7.07 / 6.66	0.97 / 7.74 / 6.58	3.71 / 9.13 / 9.39	3.94 / 9.17 / 8.78
131 072	0.84 / 6.93 / 6.94	0.88 / 6.95 / 7.31	4.04 / 9.35 / 9.02	3.53 / 9.14 / 8.88
1 048 576	0.83 / 7.15 / 6.71	0.86 / 7.37 / 6.71	3.62 / 9.53 / 8.83	3.58 / 9.43 / 9.19

A 64-item row was measured and excluded as timer-bound. At 1 024 items the table loses in latency, warm (10.99 vs 9.33) and above all cold (21.83 vs 9.64): with that few operations, re-warming the evicted 64 KB is not amortised. From 16 384 items up the table wins in every column. MUL and DEC are within run-to-run noise of each other (single cells such as 9.33 at 24 576 warm are noise, not a trend). Beta rustc 1.100.0-beta.4 gives the same shape with DEC about 1 ns faster in latency (8.2–8.5 ns/op), ratios 2.0–2.4×.